

# Listas em Python — Exercícios de Engenharia

*append · remove · copy · extend*

*Engenharia Mecânica · Elétrica · Telecomunicações*

Nome: \_\_\_\_\_

Data: \_\_\_\_ / \_\_\_\_ / \_\_\_\_

Turma / Curso: \_\_\_\_\_

Nota: \_\_\_\_\_

## Referência rápida — métodos de lista:

| Método                         | Sintaxe                               | O que faz                               |
|--------------------------------|---------------------------------------|-----------------------------------------|
| <code>lista.append(x)</code>   | <code>lista.append(42.5)</code>       | Adiciona x ao final da lista            |
| <code>lista.remove(x)</code>   | <code>lista.remove(42.5)</code>       | Remove a 1ª ocorrência de x             |
| <code>lista.pop(i)</code>      | <code>lista.pop(0)</code>             | Remove e retorna o elemento no índice i |
| <code>lista.pop()</code>       | <code>lista.pop()</code>              | Remove e retorna o último elemento      |
| <code>lista.extend(L)</code>   | <code>lista.extend([1.0, 2.0])</code> | Acrescenta todos os itens de L ao final |
| <code>lista.copy()</code>      | <code>b = lista.copy()</code>         | Cria cópia independente da lista        |
| <code>lista[:]</code>          | <code>b = lista[:]</code>             | Cópia via fatiamento — mesmo efeito     |
| <code>lista.insert(i,x)</code> | <code>lista.insert(0, 99.9)</code>    | Insere x na posição i                   |
| <code>lista.index(x)</code>    | <code>lista.index(42.5)</code>        | Retorna índice da 1ª ocorrência de x    |
| <code>len(lista)</code>        | <code>len(lista)</code>               | Retorna o número de elementos           |

## Bloco 1 — `append()` Adicionar elementos à lista

|       |                     |                                                             |        |
|-------|---------------------|-------------------------------------------------------------|--------|
| Ex. 1 | <code>append</code> | Eng. Mecânica — Registro de temperatura de forno industrial | básico |
|-------|---------------------|-------------------------------------------------------------|--------|

Um forno industrial é monitorado a cada hora durante um turno de 6 horas. Escreva um programa que:

- Crie uma lista vazia chamada `temperaturas`
- Use `append()` para inserir as 6 leituras fornecidas abaixo
- Exiba a lista completa, o valor máximo e o valor mínimo

### Dados de entrada

`leituras (°C): 820, 835, 848, 791, 812, 856`

### Saída esperada:

```
» Leituras: [820, 835, 848, 791, 812, 856]
Máximo: 856 °C
Mínimo: 791 °C
```

|       |        |                                                            |        |
|-------|--------|------------------------------------------------------------|--------|
| Ex. 2 | append | Eng. Elétrica — Coleta de amostras de tensão em barramento | básico |
|-------|--------|------------------------------------------------------------|--------|

Um osciloscópio registra amostras de tensão em um barramento de 12 V. Use `append()` dentro de um laço `for` para acumular as leituras em uma lista. Ao final, calcule a média e exiba alerta para qualquer leitura fora da faixa  $\pm 10\%$  (entre 10,8 V e 13,2 V).

## Datos de entrada

```
amostras (V): 11.9, 12.1, 12.0, 13.5, 11.7, 12.3, 10.5, 12.0
```

**Saída esperada:**

```
» Média: 12.01 V
ALERTA: 13.5 V fora da faixa!
ALERTA: 10.5 V fora da faixa!
```

 **Dica:** Itere sobre a lista de amostras com for e use `append()` a cada iteração para construir a lista de leituras.

*Código Python:*

---



---

|          |        |                                                     |               |
|----------|--------|-----------------------------------------------------|---------------|
| Ex.<br>3 | append | Eng. Telecomunicações — Buffer de pacotes recebidos | intermediário |
|----------|--------|-----------------------------------------------------|---------------|

Um roteador recebe pacotes com tamanhos variados (em bytes). Use `append()` dentro de um laço para armazená-los em um buffer e calcule:

- Total de pacotes recebidos
- Tamanho total em bytes
- Porcentagem de pacotes acima de 1500 bytes (pacotes jumbo)

| Dados de entrada                                                         |
|--------------------------------------------------------------------------|
| tamanhos (bytes): 512, 1024, 1500, 1518, 256, 1500, 2048, 64, 1500, 1024 |

Saída esperada:

|                                                                                     |                                                                                                                   |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| »                                                                                   | Pacotes recebidos : 10<br>Total de bytes : 10946<br>Pacotes jumbo : 1 (10.0%)                                     |
|  | <b>Dica:</b> Conte os pacotes jumbo com um acumulador ou com <code>sum()</code> e uma condição dentro do gerador. |

Código Python:

## Bloco 2 — remove() · pop() Remover elementos da lista

Ex.  
4

remove

Eng. Mecânica — Descarte de leituras com defeito de sensor

básico

Um acelerômetro gera o valor sentinela `-999.0` quando há falha de comunicação. Escreva um programa que remova todos os valores `-999.0` da lista usando `remove()` em laço `while` e exiba a lista limpa com a quantidade de leituras válidas.

### Dados de entrada

vibracao (m/s<sup>2</sup>): 0.12, 0.34, -999.0, 0.28, 0.91, -999.0, 1.42, 0.67

### Saída esperada:

» Leituras válidas: [0.12, 0.34, 0.28, 0.91, 1.42, 0.67]  
Quantidade : 6



**Dica:** `remove()` elimina apenas a primeira ocorrência — use `while (-999.0 in lista)` para eliminar todas.

Código Python:

Ex.  
5

remove

Eng. Elétrica — Gerenciamento de cargas no  
sistema elétrico

intermediário

Um sistema de gerenciamento de energia mantém uma lista de cargas ativas em kW. Quando uma carga é desligada, ela é removida. Escreva um programa que:

- Remova a carga de 8.0 kW (desligada por sobrecarga)
- Remova a carga de 40.0 kW (equipamento em manutenção)
- Exiba a potência total antes e depois, e a redução obtida

#### Dados de entrada

cargas (kW): 15.0, 22.5, 8.0, 40.0, 12.5, 30.0

#### Saída esperada:

```
>> Cargas ativas : [15.0, 22.5, 12.5, 30.0]
    Potência antes: 128.0 kW
    Potência agora: 80.0 kW
    Redução       : 48.0 kW
```

Código Python:



## Bloco 3 — copy() Copiar listas com independência

|          |      |                                                                  |        |
|----------|------|------------------------------------------------------------------|--------|
| Ex.<br>7 | copy | Eng. Mecânica — Backup de parâmetros de usinagem antes de ajuste | básico |
|----------|------|------------------------------------------------------------------|--------|

Antes de ajustar os parâmetros de velocidade de corte (m/min) de uma máquina CNC, o operador deve preservar os valores originais. Escreva um programa que:

- Crie um backup com `copy()` antes de qualquer modificação
- Altere o primeiro valor da lista original para 130.0
- Exiba ambas as listas e explique (em comentário no código) por que o backup não foi alterado
- Mostre também o que aconteceria se tivesse usado `=` em vez de `copy()`

## Datos de entrada

```
params_originais (m/min): 120.0, 150.0, 95.0, 200.0, 110.0
novo valor para índice 0 : 130.0
```

## Saída esperada:

|   |                                                                                                                                                     |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| » | Original (modificada): [130.0, 150.0, 95.0, 200.0, 110.0]<br>Backup (preservado): [120.0, 150.0, 95.0, 200.0, 110.0]                                |
| 💡 | <b>Dica:</b> lista_b = lista_a não cria uma cópia — cria um segundo nome para o mesmo objeto em memória. Qualquer alteração aparece nos dois nomes. |

### Código Python:

*Código Python:*



*Código Python:*

## Bloco 4 — extend() Concatenar listas

|        |        |                                                                           |        |
|--------|--------|---------------------------------------------------------------------------|--------|
| Ex. 10 | extend | Eng. Mecânica — Consolidação de leituras de múltiplos sensores de pressão | básico |
|--------|--------|---------------------------------------------------------------------------|--------|

Três sensores de pressão monitoram diferentes pontos de uma tubulação. As leituras estão em listas separadas. Use `extend()` para consolidar tudo em uma única lista e calcule média, máximo e mínimo do sistema.

| Datos de entrada |                           |
|------------------|---------------------------|
| sensor_A         | (bar): 3.2, 3.4, 3.1, 3.5 |
| sensor_B         | (bar): 4.1, 4.0, 4.3, 3.9 |
| sensor_C         | (bar): 2.8, 2.9, 3.0, 2.7 |

### Saída esperada:

|                                                                                   |                                                                                                                                                                                                                                      |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| »                                                                                 | Total de leituras: 12<br>Pressão média : 3.41 bar<br>Pressão máxima : 4.3 bar<br>Pressão mínima : 2.7 bar                                                                                                                            |
|  | <b>Dica:</b> <code>extend()</code> é diferente de <code>append()</code> : <code>append(lista)</code> insere a lista como um único elemento, criando lista aninhada. <code>extend(lista)</code> insere cada elemento individualmente. |

### Código Python:

|        |        |                                                                 |               |
|--------|--------|-----------------------------------------------------------------|---------------|
| Ex. 11 | extend | Eng. Elétrica — Fusão e análise de medições de dois multímetros | intermediário |
|--------|--------|-----------------------------------------------------------------|---------------|

Dois técnicos mediram a resistência de uma bobina com multímetros diferentes. Una as medições com `extend()`, ordene a lista resultante, calcule a média e identifique valores que desviam mais de  $\pm 15\%$  da média.

## Datos de entrada

medidor\_1 ( $\Omega$ ): 48.2, 51.0, 49.7, 47.5, 52.3

medidor\_2 ( $\Omega$ ): 50.1, 46.8, 53.0, 49.0, 55.2

**Saída esperada:**

```
>> Medições ordenadas: [46.8, 47.5, 48.2, 49.0, 49.7, 50.1, 51.0, 52.3, 53.0, 55.2]
```

Média: 50.28  $\Omega$  | Tolerância:  $\pm 7.54 \Omega$

(nenhum valor fora da faixa neste conjunto)



**Dica:** Use `medidor_1.copy()` antes de `extend()` para não modificar a lista original do medidor 1.

*Código Python:*

## Ex. 12

## extend

## Eng. Telecomunicações — Agregação de logs de potência de antenas 5G

**avançado**

Três antenas 5G registram a potência do sinal recebido (RSRP em dBm). Use os logs com `extend()` e calcule:

- Taxa de cobertura adequada (leituras  $\geq -100$  dBm)
- Índice e valor do pior sinal registrado
- Média de todas as leituras

## Datos de entrada

```
antena_norte (dBm): -85, -90, -105, -78, -92, -110, -88
antena_sul    (dBm): -95, -88, -102, -79, -85, -91, -97
antena_leste  (dBm): -80, -75, -108, -93, -86, -103, -82
limiar de cobertura: -100 dBm
```

### Saída esperada:

```
>> Total de amostras      : 21
    Em cobertura (≥-100) : 16
    Taxa de cobertura    : 76.2%
    Pior sinal           : -110 dBm (índice 5)
    Média                 : -91.4 dBm
```

*Código Python:*

## Bloco 5 — Exercícios integrados `append · remove · copy · extend`

## Ex. 13

## append + remove

**Eng. Mecânica — Controle de qualidade de peças usinadas**

**intermediário**

Um lote de 10 peças é inspecionado. Use `append()` para construir separadamente as listas de aprovadas e reprovadas a partir das medições fornecidas, com base na tolerância indicada.

## Dados de entrada

```

medições (mm): 50.02, 49.97, 50.06, 49.94, 50.00,
                50.08, 49.99, 50.01, 49.92, 50.04
diâmetro nominal : 50.00 mm
tolerância       : ±0.05 mm

```

### Saída esperada:

```
» Aprovadas (7): [50.02, 49.97, 50.0, 49.99, 50.01, 50.04, ...]
Reprovadas (3): [50.06, 50.08, 49.92]
Índice de aprovação: 70%
```



**Dica:** Use `abs(medida - nominal) <= tolerancia` para verificar se a peça está dentro da faixa.

*Código Python:*

Um engenheiro analisa dois cenários de carga em uma subestação: o atual e um cenário futuro com novos equipamentos. Use `copy()` para preservar o cenário atual e `extend()` para compor o futuro. Exiba um resumo comparativo com total, média e maior carga de cada cenário.

## Dados de entrada

```
cenário atual (kW): 15.0, 22.5, 8.0, 40.0, 12.5, 30.0
novas cargas (kW): 55.0, 18.0, 75.0
```

**Saída esperada:**

```
» --- Cenário atual ---
    Total : 128.0 kW | Média : 21.3 kW | Maior : 40.0 kW
--- Cenário futuro ---
    Total : 276.0 kW | Média : 30.7 kW | Maior : 75.0 kW
```

*Código Python:*

## Ex. 15

**integrado**

## Eng. Telecomunicações — Filtragem de ruído em sinal de dois canais

**avançado**

Um sistema de processamento de sinal recebe amostras de dois canais (I e Q). Escreva um programa que:

- Uma os dois canais com extend()

- Guarde um backup com `copy()` antes de filtrar
- Calcule média ( $\mu$ ) e desvio padrão ( $\sigma$ ) do sinal unido
- Remova com `remove()` todos os valores fora de  $\mu \pm 3\sigma$  (ruído)
- Exiba sinal bruto, ruídos removidos e sinal limpo

## Datos de entrada

```
canal_I: 0.82, 0.79, 0.85, 5.20, 0.81, 0.78, 0.83 ← 5.20 é ruído
canal_Q: 0.41, 0.39, 0.43, 0.40, -4.80, 0.42, 0.38 ← -4.80 é ruído
σ = sqrt( Σ(xi - μ)² / N )      (desvio padrão amostral simples)
```

### Saída esperada:

```
>> Sinal bruto (14 amostras): [0.82, 0.79, 0.85, 5.2, 0.81, ..., -4.8,
0.42, 0.38]
Ruídos removidos : [5.2, -4.8]
Sinal limpo (12 amostras): [0.82, 0.79, 0.85, 0.81, 0.78, 0.83, 0.41,
...]
```



**Dica:** Monte uma lista auxiliar com os ruídos antes de removê-los — nunca remova dentro do mesmo for que percorre a lista.

*Código Python:*

